

Microservices Architecture

Microservices architecture mein ek large application ko multiple small and independently deployable services mein divide kiya jata hai. Har service ek specific business capability handle karti hai aur services APIs ya asynchronous messaging ke through communicate karti hain. For example, mere insurance platform mein maine Claim Service aur Policy Service ko separate microservices ke roop mein implement kiya. Claim Service claim creation handle karti hai, jabki Policy Service policy information aur validation handle karti hai. Dono services ki separate codebase, Docker image, ECR repository aur ECS Fargate deployment hai. Isliye main unhe independently deploy aur scale kar sakta hoon. Microservices generally small, loosely coupled services hote hain jo well-defined APIs ke through communicate karte hain aur independently develop, deploy aur scale kiye ja sakte hain.

Monolithic aur Microservices Architecture mein kya difference hai?

Monolithic architecture mein application ke saare modules ek single codebase aur deployment unit ka part hote hain. Agar ek small change bhi karna ho, toh complete application ko build, test aur deploy karna pad sakta hai.

Microservices architecture mein application ko independent services mein divide kiya jata hai. Har service ko separately develop, deploy aur scale kiya ja sakta hai.

For example, agar mujhe Policy Service mein change karna hai, toh main Claim Service ko redeploy kiye bina sirf Policy Service ka new container version deploy kar sakta hoon.

Lekin microservices operational complexity bhi introduce karti hain, jaise network failures, service discovery, distributed logging, tracing aur data consistency.

Microservices ek dusre se kaise communicate karti hain?

Microservices synchronous aur asynchronous dono tarah se communicate kar sakti hain.

Synchronous communication mein ek service doosri service ko REST API, HTTP ya gRPC call karti hai aur response ka wait karti hai.

Asynchronous communication mein services message broker ya event service, jaise Amazon SQS, SNS, EventBridge ya Kafka ke through communicate karti hain.

Mere project mein Claim Service Policy Service ko REST API ke through call karke policy validate karti hai. Yeh synchronous communication hai, kyunki claim creation se pehle validation result required hai.

Notification ya analytics jaise use cases ke liye main asynchronous event-driven communication prefer karta hoon, kyunki calling service ko notification complete hone ka wait nahi karna padta.

Synchronous aur Asynchronous Communication mein kya difference hai?

Synchronous communication mein caller service response ka wait karti hai. Isliye response immediate milta hai, lekin downstream service unavailable ho toh request fail ya timeout ho sakti hai.

Asynchronous communication mein producer message ya event publish karta hai aur apna processing continue karta hai. Consumer baad mein message process karta hai.

For example, claim create karne se pehle policy validation immediate required hai, isliye synchronous API call suitable hai.

Lekin claim create hone ke baad customer notification send karna asynchronous ho sakta hai. Claim Service SQS ya EventBridge mein event publish kar sakti hai aur Notification Service us event ko process kar sakti hai.

Agar ek microservice unavailable ho jaye toh kya hoga?

Microservices design mein downstream failure ko gracefully handle karna important hai. Main timeout, retry with exponential backoff, circuit breaker aur fallback patterns use karta hoon.

Mere project mein jab Policy Service unavailable thi, tab Claim Service ne controlled 503 Service Unavailable response return kiya. Isse uncontrolled application crash ya incorrect claim creation prevent hui.

Retries sirf transient errors ke liye limited count ke saath configure honi chahiye. Excessive retries cascading failure create kar sakti hain.

Agar continuous failures mil rahe hain, circuit breaker downstream calls temporarily stop kar deta hai. Isse system ko recover hone ka time milta hai.

Important Patterns

Timeout

Limited retries

Exponential backoff

Circuit breaker

Fallback

Dead-letter queue

Health checks

Monitoring and alerting

Har microservice ka separate database kyon hona chahiye?

Ideally, each microservice should own its data. Ek service ko directly doosri service ke database tables access nahi karne chahiye.

For example, Claim Service claim data own karegi aur Policy Service policy data own karegi. Claim Service ko policy information chahiye toh woh Policy Service API ya events use karegi, directly Policy database query nahi karegi. Isse loose coupling, independent deployment aur clear ownership maintain hota hai.

Lekin is model mein cross-service transactions aur data consistency difficult ho sakti hai. Iske liye Saga pattern, events aur eventual consistency use ki ja sakti hai.

AWS guidance bhi data stores, asynchronous communication aur data consistency ko microservices ke important cross-service design considerations ke roop mein identify karti hai.

API Gateway ka microservices mein kya role hai?

API Gateway clients ke liye single entry point provide karta hai. Client ko har individual microservice ka address know karne ki requirement nahi hoti.

API Gateway authentication, authorization, rate limiting, throttling, request validation, routing aur API monitoring handle kar sakta hai.

For example:

/claims request Claim Service ko route hoti hai.

/policies request Policy Service ko route hoti hai.

Mere container-based implementation mein ALB path-based routing use hui. Serverless architecture mein similar client-facing API management ke liye Amazon API Gateway use kiya ja sakta hai.

Microservices mein CI/CD kaise work karta hai?

Ideally each microservice ki independent CI/CD pipeline honi chahiye.

Developer ke code push ya pull request ke baad pipeline unit testing, code quality checks, security scanning, Docker build, image scanning aur deployment validation perform karti hai.

Mere implementation mein Ruff, Pytest, Bandit, Gitleaks, Checkov, SonarCloud, Trivy aur SBOM generation jaise checks use hue. Successful validation ke baad Docker image ECR mein push hoti hai aur ECS service ka task definition update hota hai.

Infrastructure changes ke liye maine separate Terraform plan, apply aur destroy workflows use kiye. AWS authentication ke liye GitHub OIDC aur temporary STS credentials use kiye, long-lived AWS keys nahi.

Microservices ko secure kaise karte hain?

Main microservices security ko multiple layers mein implement karta hoon.

External traffic ke liye TLS, authentication, authorization, WAF aur rate limiting use karta hoon. Services ko private subnets mein deploy karta hoon aur security groups ke through only required traffic allow karta hoon.

Service-to-service communication ke liye least-privilege access, token-based authentication, IAM roles aur, where required, mutual TLS use kiya ja sakta hai.

Secrets ko source code ya Docker image mein store nahi karta. Unhe AWS Secrets Manager ya Systems Manager Parameter Store mein maintain karta hoon.

CI/CD mein source code scanning, dependency scanning, IaC scanning, secrets scanning aur container image scanning use karta hoon.

Readiness aur liveness mein difference?

Liveness checks whether the application process is alive. Readiness checks whether it is ready to receive traffic.

Which microservices design patterns have you used?

A. Database per Service Pattern:

I follow the Database per Service pattern, where every microservice owns its data.

Claim Service owns claim data and Policy Service owns policy data. Claim Service cannot directly access Policy Service database tables. It must use the Policy Service API or consume policy-related events.

This avoids tight coupling and allows both services to change and deploy independently.

B. API Gateway Pattern:

API Gateway provides a common entry point for clients. It handles concerns such as routing, authentication, authorization, throttling, request validation and monitoring.

For example:

/claims routes to Claim Service

/policies routes to Policy Service

In my container-based implementation, I used an Application Load Balancer with path-based routing. For a serverless implementation, I can use Amazon API Gateway with Lambda.

C. Circuit Breaker Pattern

Circuit Breaker prevents repeated calls to a failing downstream service.

If Policy Service continuously fails, Claim Service should not keep sending calls for every incoming request. After a failure threshold, the circuit opens and Claim Service returns a controlled response.

After a configured time, the circuit enters the half-open state and sends a limited test request. If the request succeeds, normal communication resumes.

Three states

Closed: Requests flow normally.

Open: Requests are blocked because the dependency is failing.

Half-open: A test request checks whether the dependency has recovered.

D. Retry and Timeout Pattern

Every service-to-service API call should have a timeout. Otherwise, the calling service can wait indefinitely and exhaust its resources.

I use limited retries with exponential backoff only for transient failures such as temporary network issues or selected 5xx errors.

I do not blindly retry validation failures such as 400 Bad Request or 404 Not Found, because retries will not correct invalid input.

Strong line

Timeout protects the caller, retries handle temporary failures, and circuit breaker prevents cascading failure.

E. Saga Pattern

Saga Pattern manages a business transaction that spans multiple microservices. Instead of one distributed database transaction, each service performs a local transaction.

If a later step fails, compensating actions reverse or correct the previous business operations.

Insurance example

Claim Service creates a claim with PENDING status.

Policy Service validates the policy.

Payment Service processes the deductible.

Document Service stores claim documents.

Notification Service sends confirmation.

If payment processing fails, the previous database transaction cannot simply be rolled back across all services. A compensating action updates the claim to PAYMENT_FAILED or CANCELLED.

Saga types

Choreography: Services react to one another's events.

Orchestration: A central orchestrator controls the workflow.

What I would say about selection

For a small number of simple steps, choreography can work well. For a complex insurance claim workflow with many decisions and compensating actions, I prefer orchestration using AWS Step Functions because the workflow is easier to visualize, monitor and troubleshoot.

How do you design APIs for microservices?

1. Resource-based URLs

URLs should represent business resources:

POST /v1/claims

GET /v1/claims/{claimId}

PUT /v1/claims/{claimId}

PATCH /v1/claims/{claimId}/status

GET /v1/policies/{policyId}

2. Correct HTTP Methods

- GET: Retrieve data
- POST: Create a resource
- PUT: Fully replace or update a resource
- PATCH: Partially update a resource
- DELETE: Delete a resource

3. Standard HTTP Status Codes:

200 OK Request successful

201 Created New resource created

400 Bad Request Invalid request format

401 Unauthorized Authentication missing or invalid

403 Forbidden User is authenticated but not authorized

404 Not Found Resource does not exist

409 Conflict Duplicate or conflicting operation

422 Unprocessable Entity Business validation failed

429 Too Many Requests Rate limit exceeded
500 Internal Server Error Unexpected service failure
503 Service Unavailable Dependency temporarily unavailable
504 Gateway Timeout Dependency did not respond in time

Synchronous vs Event-Driven Communication

Use synchronous API when:

Immediate response is required

User is waiting

Simple request-response flow exists

Immediate validation is required

Use asynchronous events when:

Immediate response is not required

Multiple consumers need the same information

Background processing is suitable

Producer and consumer should be loosely coupled

Temporary consumer unavailability should be tolerated

Microservices 12 factor principles

Haan bhai, **12-Factor App principles microservices aur cloud-native applications ke interviews mein frequently pooche jaate hain**. Ye exclusively microservices ke liye nahi bane the, balki scalable Software-as-a-Service applications ke liye methodology hai. Lekin practically, modern microservices ko design aur deploy karte waqt ye principles widely apply kiye jaate hain.

12-Factor App ka basic meaning

12-Factor methodology batati hai ki application ya microservice ko kaise design karein taaki woh:

- Cloud par easily deploy ho
- Horizontally scale ho sake
- DEV, QA aur PROD mein consistently run ho
- Containers aur Kubernetes ke saath compatible ho
- Configuration code se separate rahe
- Failure ke baad easily restart ho
- CI/CD ke through reliably deploy ho

Official Twelve-Factor methodology portability, deployment automation, DEV-PROD parity, continuous deployment aur scalable execution ko encourage karti hai. [\[12factor.net\]](https://12factor.net)

Tumhare insurance project mein **Claim Service** aur **Policy Service**, dono independent 12-factor applications/microservices ki tarah design ki ja sakti hain.

12-Factor principles ka overview

1. Codebase
2. Dependencies
3. Config
4. Backing Services

5. Build, Release, Run
6. Processes
7. Port Binding
8. Concurrency
9. Disposability
10. Dev/Prod Parity
11. Logs
12. Admin Processes

1. Codebase

Principle

One codebase tracked in version control, many deploys.

Har microservice ka source code version control, jaise Git, mein maintain hona chahiye. Ek codebase ko different environments mein deploy kiya ja sakta hai. [\[12factor.net\]](https://12factor.net/), [\[en.wikipedia.org\]](https://en.wikipedia.org/)

Insurance example:

Claim Service Git repository

├── DEV deployment

├── QA deployment

└── PROD deployment

Codebase same hai, lekin deployments multiple hain.

Environment-specific code branches maintain nahi karni chahiye:

claim-service-dev-code

claim-service-qa-code

claim-service-prod-code

Preferred approach:

Same Claim Service code

↓

Different configuration

↓

DEV, QA and PROD deployments

Interview line

I maintain one version-controlled codebase for each independently deployable microservice, and I deploy the same code artifact across DEV, QA and PROD using environment-specific configuration.

2. Dependencies

Principle

Explicitly declare and isolate dependencies.

Application ki dependencies explicitly declare honi chahiye. Application ko ye assume nahi karna chahiye ki required package ya library underlying server par already installed hogi. [\[12factor.net\]](https://12factor.net/), [\[geeksforgeeks.org\]](https://www.geeksforgeeks.org/)

Python Claim Service:

requirements.txt

fastapi==0.115.0

uvicorn==0.30.0

requests==2.32.0

Node.js frontend:

package.json

package-lock.json

Container build:

RUN pip install --no-cache-dir -r requirements.txt

Isse dependency versions consistent rahengi.

Interview line

I declare all application dependencies explicitly in files such as requirements.txt or package.json, and I package them inside the container image so the service does not depend on libraries installed on the ECS host.

3. Config

Principle

Store configuration in the environment.

Environment-specific configuration source code mein hardcode nahi karni chahiye. Configuration ko environment variables ya external configuration service ke through inject karna chahiye. [\[12factor.net\]](https://12factor.net/), [\[geeksforgeeks.org\]](https://www.geeksforgeeks.org/)

Wrong:

```
POLICY_SERVICE_URL = "http://dev-alb/policies"
```

Correct:

```
import os
```

```
POLICY_SERVICE_URL = os.getenv("POLICY_SERVICE_URL")
```

ECS task definition:

```
environment = [
```

```
{
```

```
name = "POLICY_SERVICE_URL"
```

```
value = var.policy_service_url
```

```
}]
```

Secrets ke liye:

AWS Secrets Manager

AWS Systems Manager Parameter Store

Insurance example

Same Claim Service image

↓

DEV: POLICY_SERVICE_URL=dev-policy.internal

QA: POLICY_SERVICE_URL=qa-policy.internal

PROD: POLICY_SERVICE_URL=prod-policy.internal

Interview line

I keep configuration separate from application code. I inject environment-specific values through ECS task definitions and store sensitive values in Secrets Manager instead of hardcoding them.

4. Backing Services

Principle

Treat backing services as attached resources.

Database, message queue, cache, object storage aur third-party services ko loosely coupled attached resources treat karna chahiye.

Examples:

RDS PostgreSQL

DynamoDB

Amazon S3

Amazon SQS

Amazon ElastiCache

External Policy API

Application ko database server ki fixed identity par tightly dependent nahi hona chahiye.

Claim Service

↓

DATABASE_URL environment variable

↓

DEV database / QA database / PROD database

Database replace hone par ideally application code change nahi hoga. Sirf configuration update hogi.

Interview line

I treat databases, queues, caches and storage as replaceable backing services. The microservice connects to them using externally supplied endpoints and credentials.

5. Build, Release, Run

Principle

Strictly separate build, release and run stages.

Application lifecycle ko teen separate stages mein divide karna chahiye.

Build

↓

Release

↓

Run

Build stage

Code se immutable artifact create hota hai:

Source code

↓

Docker build

↓

Container image

↓

Push to Amazon ECR

Release stage

Built image ko environment configuration ke saath combine kiya jata hai:

ECR image + ECS task definition + environment config

Run stage

Released version production environment mein execute hoti hai:

ECS Service

↓

ECS Fargate tasks

Interview line

My CI pipeline builds a versioned container image and pushes it to ECR. The deployment pipeline combines that image with the environment-specific ECS task definition, and ECS runs that immutable release.

6. Processes

Principle

Execute the application as one or more stateless processes.

Application process stateless hona chahiye. Persistent data container memory ya local filesystem mein store nahi karni chahiye.

Tumhare project ka earlier design:

claims = []

Ye claim data application memory mein rakhta tha.

Problem:

ECS task restart

↓

Memory reset

↓

Claim data lost

Correct design:

Claim Service container

↓

Data stored in RDS/DynamoDB

Sessions ke liye:

ElastiCache Redis

DynamoDB

Database

Twelve-factor processes stateless aur share-nothing hote hain. Persistent data backing service mein store honi chahiye, kyunki future requests kisi doosre process/task ko mil sakti hain.

Interview line

I design microservices as stateless processes. I do not rely on local container memory or filesystem for persistent business data. Claims are stored in a database so any ECS task can process the next request.

7. Port Binding

Principle

Export services through port binding.

Microservice ko self-contained hona chahiye aur apni functionality network port ke through expose karni chahiye.

Example:

Claim Service → Port 8000

Policy Service → Port 8001

Dockerfile:

EXPOSE 8000

FastAPI:

uvicorn app.main:app --host 0.0.0.0 --port 8000

ECS:

ALB Target Group

↓

ECS Task IP:8000

Interview line

Each microservice exposes its API through a configured application port. The ECS task exposes the container port, and the ALB target group routes traffic to that port.

8. Concurrency

Principle

Scale out through the process model.

Load badhne par ek server ko continuously bada karne ke bajaye multiple process or container instances run karo.

Vertical scaling:

1 ECS task

CPU: 256 → 1024

Horizontal scaling:

Claim Task 1

Claim Task 2

Claim Task 3

Claim Task 4

AWS implementation:

ALB

↓

Multiple ECS Fargate tasks

↓

ECS Service Auto Scaling

EKS implementation:

Kubernetes Deployment

↓

Multiple pods

↓

Horizontal Pod Autoscaler

Interview line

I scale microservices horizontally by increasing ECS tasks or Kubernetes pods. Because services are stateless, any instance can process a request.

9. Disposability

Principle

Maximize robustness through fast startup and graceful shutdown.

Microservice instances ko quickly start aur gracefully stop hona chahiye. Isse deployment, auto scaling aur failure recovery reliable hoti hai.

ECS example:

Task unhealthy

↓

ECS stops old task

↓

New task starts

↓

ALB health check passes

↓

Traffic starts

Graceful shutdown ke time application ko:

- New requests lena stop karna chahiye
- Existing request complete karni chahiye
- Database connection close karna chahiye
- In-progress message safely handle karna chahiye

Interview line

My containers start quickly and handle termination signals gracefully. ECS removes an unhealthy task from the load balancer and replaces it without depending on local state.

10. Dev/Prod Parity

Principle

Keep development, staging and production as similar as possible.

DEV, QA aur PROD ke architecture aur dependencies ko similar rakhna chahiye.

Bad approach:

DEV → SQLite + local server

PROD → PostgreSQL + container platform

Better approach:

DEV → Docker + PostgreSQL

QA → Docker + PostgreSQL

PROD → Docker + PostgreSQL

Capacity aur instance size different ho sakte hain:

DEV RDS → smaller instance

PROD RDS → Multi-AZ larger instance

Lekin technology aur deployment model reasonably same hone chahiye.

Interview line

I maintain DEV, QA and PROD parity by deploying the same container artifact and using the same core backing-service technologies. Only capacity, scaling and environment configuration differ.

11. Logs

Principle

Treat logs as event streams.

Application ko log files locally manage nahi karni chahiye. Application logs standard output and standard error par emit kare, aur platform unhe collect kare.

Application:

```
print("Claim created successfully")
```

Container flow:

Application stdout/stderr

↓

ECS awslogs driver

↓

CloudWatch Logs

↓

Metric filters and alarms

Advanced implementation:

Structured JSON logging

Correlation ID

Request ID

Claim ID

Service name

Log level

Timestamp

Interview line

My microservices write structured logs to stdout and stderr. ECS forwards those logs to CloudWatch Logs, where I configure search, metrics, dashboards and alarms.

12. Admin Processes

Principle

Run administrative tasks as one-off processes.

Database migration, data correction aur operational scripts ko one-time controlled process ke roop mein run karna chahiye. Code same repository aur same application release ke saath versioned hona chahiye.

Examples:

Database schema migration

One-time data correction

Seed data loading

Expired claims cleanup

ECS example:

Same Claim Service image

↓

One-off ECS task

↓

Database migration command

EKS example:

Kubernetes Job

Interview line

I run database migrations and operational scripts as controlled one-off ECS tasks or Kubernetes jobs using the same codebase and release artifact as the application.

Insurance project mapping

Tumhare project mein mapping aise hogi:

Codebase → Claim and Policy services maintained in Git
Dependencies → requirements.txt and container images
Config → ECS environment variables and Secrets Manager
Backing services → RDS, DynamoDB, S3, SQS and Policy API
Build, Release, Run → GitHub Actions, ECR and ECS deployment
Processes → Stateless ECS tasks
Port Binding → Claim and Policy service ports behind ALB
Concurrency → Multiple ECS tasks through Service Auto Scaling
Disposability → Health checks, task replacement and graceful shutdown
Dev/Prod Parity → Same container image promoted across environments
Logs → stdout/stderr forwarded to CloudWatch Logs
Admin Processes → One-off ECS database migration task

Important interview clarification

Agar interviewer kahe:

Are 12-factor principles microservices design principles?

Best answer:

The Twelve-Factor App methodology was originally defined for building scalable software-as-a-service applications, not specifically for microservices. However, these principles align very well with cloud-native microservices because each microservice is independently built, configured, deployed, scaled and operated as an application. The methodology focuses on how an individual service should behave, while additional microservice patterns are required for inter-service communication, distributed transactions, resiliency and data consistency.

12-Factor methodology individual application/service ki deployability aur runtime behaviour par focus karti hai. Ye distributed transactions, service-to-service failure handling aur complete microservices system design ka replacement nahi hai.

Additional microservices topics:

API Gateway

Service discovery

Circuit breaker

Retry and timeout

Saga pattern

Event-driven architecture

Distributed tracing

Database per service

Idempotency

Dead-letter queue

Observability

Interview-ready 2-minute answer

Yes, I follow the Twelve-Factor App principles while designing cloud-native microservices. Although the methodology was originally created for Software-as-a-Service applications, it aligns well with independently deployable microservices.

I maintain one version-controlled codebase for each service and explicitly declare its dependencies. I keep configuration outside the code and inject environment-specific values through ECS task definitions, while sensitive configuration is stored in Secrets Manager.

I treat databases, queues, caches and storage as external backing services. My CI pipeline builds an immutable container image, pushes it to ECR, and the deployment pipeline releases the same image across DEV, QA and PROD.

I design ECS tasks as stateless processes, expose each service through port binding, and scale horizontally by increasing the number of tasks. Containers support fast startup and graceful shutdown so ECS can replace unhealthy tasks safely.

The services write structured logs to stdout and stderr, which ECS forwards to CloudWatch Logs. Database migrations and administrative jobs are executed as controlled one-off ECS tasks.

These principles improve portability, scalability, deployment consistency and operational reliability. For complete microservices architecture, I additionally use patterns such as retries, timeouts, circuit breakers, event-driven communication and distributed tracing.

Short memory trick

Code, Dependencies, Config, Services

Build, Process, Port, Scale

Dispose, Parity, Logs, Admin

Abhi ka actionable step

Sabse pehle project ke ye four principles yaad karo, kyunki tumhare existing ECS project se directly connect hote hain:

Config → Environment variables / Secrets Manager

Processes → Stateless ECS tasks

Concurrency → Multiple ECS tasks

Logs → CloudWatch Logs

